

Aula 04 - Colaboração Avançada na Nuvem e Fluxos de Organização

Descrição: Capacitar os estudantes a contribuírem para projetos protegidos utilizando o fluxo de **Forks e Pull Requests (PRs)**, gerenciarem estados temporários de código com **Stash e Clean**, e manterem históricos de commits limpos e lineares através do **Rebase**.

A. Forks e Pull Requests (PRs): O Padrão de Colaboração do Mercado

Nas aulas anteriores, trabalhamos como colaboradores diretos em um repositório central. No entanto, em grandes projetos ou contribuições de código aberto, desenvolvedores não possuem permissão para fazer **git push** diretamente na branch principal (**main/master**). Para resolver isso, utilizamos dois conceitos cruciais:

- **Fork (Bifurcação):** É uma cópia pessoal de um repositório de terceiros hospedada diretamente na *sua* conta do GitHub. Essa cópia é totalmente sua, permitindo que você faça as alterações que desejar sem afetar o projeto original.
- **Pull Request (PR - Solicitação de Puxada):** É a forma de dizer ao dono do repositório original: *"Olá! Eu fiz melhorias na minha cópia (fork) e gostaria que você as revisasse para integrá-las ao seu projeto principal."* É onde ocorrem as revisões de código (*code reviews*), debates e ajustes antes da consolidação do código.

B. Stashing e Cleaning: Gerenciando o Caos Diário

No mundo real, o trabalho de um desenvolvedor é constantemente interrompido. Imagine que você está no meio de uma tarefa complexa na sua branch, com códigos inacabados e que não compilam, quando de repente surge um bug crítico em produção que precisa de correção imediata. O que fazer?

- Você não pode fazer commit de um código quebrado e incompleto apenas para mudar de branch.
- Você não quer perder o que já escreveu.

Aqui entram as ferramentas de limpeza:

- **git stash (Guardar no Armário):** Esse comando pega todas as suas alterações não salvas (tanto na área de preparação quanto às modificações locais) e as "guarda em um armário temporário". O seu diretório de trabalho fica totalmente limpo, permitindo que você mude de branch para corrigir o bug de emergência. Depois, basta abrir o armário e recuperar seu código exatamente de onde parou.
- **git clean (Faxina de arquivos soltos):** Remove arquivos que não estão sendo rastreados pelo Git (arquivos novos que não foram adicionados com **git add**). É ideal para limpar lixos de compilação ou arquivos temporários indesejados que surgem na pasta.

C. Rebase: Reescrevendo a História de Forma Elegante

Quando trabalhamos em equipe, a branch principal (**main/master**) recebe novos commits de outros desenvolvedores o tempo todo. Se a sua branch de testes ficou desatualizada em relação à principal, você tem duas opções para integrá-las:

1. **Merge (git merge):** Cria um commit de fusão artificial (o famoso *merge commit*) para unir as histórias. O histórico do projeto fica parecendo uma teia de aranha com várias linhas se cruzando.
2. **Rebase (git rebase):** Em vez de unir as pontas com um novo commit, o Rebase pega os commits que você fez na sua branch, "recorta-os" temporariamente, puxa as atualizações mais recentes da branch principal para a base do seu projeto, e depois "cola" os seus commits de volta no topo.

Vantagem do Rebase: A linha do tempo do projeto permanece 100% linear, limpa e fácil de ler, como se você tivesse começado a trabalhar hoje em cima da última versão disponível.

Atividade Prática: O Resgate de Emergência e a Contribuição Protegida

A Situação: O repositório oficial da startup *DevTech* agora está estritamente protegido. Nenhum desenvolvedor júnior pode enviar alterações diretamente para a branch **main** do repositório do líder/instrutor.

Você recebeu a tarefa de criar uma funcionalidade chamada `contato.txt` no projeto. No entanto, enquanto estiver desenvolvendo essa melhoria, o sistema passará por um alerta de segurança (um arquivo suspeito surgirá na sua pasta) e um bug crítico de digitação na lista de colaboradores (`alunos.txt`) precisará ser corrigido com urgência. Você precisará usar o **Stash** para salvar seu progresso, o **Clean** para limpar a ameaça, o **Rebase** para alinhar seu histórico e o **Pull Request** para entregar seu trabalho!

Guia de Execução Passo a Passo

FASE 1: O Fork e o Clone

- 1. Faça o Fork:** Acesse a página do repositório `lista-turma` criado pelo líder no GitHub. No canto superior direito, clique no botão **Fork**. Escolha a sua própria conta para criar a cópia.
- 2. Clone o seu Fork:** Vá até o seu perfil do GitHub, abra o repositório que você acabou de bifurcar e copie a URL dele. No seu terminal local, clone o projeto:

```
git clone https://github.com/[SEU_USUARIO]/lista-turma.git
```
- 3. Crie a sua branch de trabalho:** Entre na pasta do projeto e crie uma branch para a nova funcionalidade:

```
git checkout -b feature-contato
```

FASE 2: O Trabalho Interrompido (Usando o Stash)

- 1. Inicie a tarefa:** Crie o arquivo `contato.txt` e adicione o seguinte texto de rascunho inacabado:
=== PÁGINA DE CONTATO DA EMPRESA ===
E-mail corporativo: contato@devtech... (escrever o resto depois)
- 2. O Alerta de Emergência:** O instrutor (ou líder) envia uma mensagem: *"Atenção equipe! Há um erro de digitação crítico no arquivo alunos.txt da branch main que precisa ser corrigido imediatamente na máquina de vocês."*
- 3. Use o Stash:** Você não pode mudar de branch com alterações inacabadas soltas sem salvar. Salve seu rascunho de forma temporária:

```
git stash
```
- 4. Verifique a limpeza:** Digite `git status`. O terminal informará que o diretório está completamente limpo. O arquivo `contato.txt` "desapareceu" temporariamente da pasta como num passe de mágica!

FASE 3: Eliminando Arquivos Suspeitos (Usando o Clean)

- 1. O ataque hacker simulado:** Um arquivo indesejado chamado `virus_simulado.log` surgiu misteriosamente na pasta do seu projeto (crie este arquivo na mão para simular).
- 2. Rode o status:** Digite `git status` e veja que ele está lá como arquivo não rastreado.
- 3. Simule a limpeza (Dry Run):** Antes de apagar de verdade, verifique o que o Git pretende remover com segurança:

```
git clean -n
```
- 4. Execute a faxina:** Tendo certeza de que é apenas o arquivo indesejado, force a exclusão permanente dele:

```
git clean -f
```

 - **Resultado:** O arquivo suspeito foi deletado do disco de forma definitiva.

FASE 4: Resolvendo a Emergência e Recuperando o Trabalho

1. Volte para a branch estável para ajudar o time:

```
git checkout master # ou main
```

2. Faça a correção rápida: Abra o arquivo `alunos.txt`, corrija qualquer erro de digitação simples ou adicione uma melhoria pontual e salve.

3. Commit a correção:

```
git add alunos.txt
```

```
git commit -m "Corrige formatação emergencial no arquivo de alunos"
```

4. Retorne para a sua branch de trabalho:

```
git checkout feature-contato
```

5. Recupere o seu rascunho do armário: Puxe seu trabalho inacabado de volta do Stash:

```
git stash pop
```

- **Resultado:** O arquivo `contato.txt` reaparece na pasta com o texto inacabado exatamente do jeito que você o deixou! Termine de escrever o arquivo (coloque um e-mail válido) e faça o commit final dele na sua branch:

```
git add contato.txt
```

```
git commit -m "Adiciona arquivo de contato oficial da DevTech"
```

FASE 5: Organizando a Linha do Tempo (Usando o Rebase)

Como você fez uma correção na branch principal enquanto sua branch de contato estava ativa, o histórico ficou bifurcado. Vamos alinhar tudo antes de enviar para aprovação.

1. Inicie o Rebase: Estando na branch `feature-contato`, diga ao Git para recolocar seus commits no topo da branch principal atualizada:

```
git rebase master # ou main
```

2. Visualize a linha do tempo perfeita:

```
git log --oneline --graph
```

3. Veja que o commit do `contato.txt` agora aparece imediatamente *depois* do commit da correção emergencial, em uma linha perfeitamente reta e organizada!

FASE 6: Enviando o Pull Request (PR)

1. Suba a branch para o seu Fork:

```
git push origin feature-contato
```

2. Abra o PR no GitHub:

- Acesse o seu repositório (fork) no site do GitHub.
- O GitHub exibirá um banner amarelo escrito: *"feature-contato had recent pushes. Compare & pull request"*. Clique em **Compare & pull request**.
- Escreva uma mensagem profissional descrevendo sua alteração: *"Este PR adiciona o arquivo de contato oficial para a empresa DevTech, além de herdar as correções feitas na linha do tempo principal."*
- Clique em **Create pull request**.

3. Revisão do Líder: O instrutor/líder abrirá a aba *Pull Requests* do repositório original, revisará o histórico limpo gerado pelo Rebase e aceitará a mesclagem!

Critérios de Sucesso (Checklist de Entrega)

Vocês saberão que concluíram a aula com êxito quando:

- [] Conseguirem ocultar e recuperar o arquivo usando `git stash` e `git stash pop`.
 - [] Eliminare o arquivo indesejado com sucesso através do comando `git clean -f`.
 - [] O histórico do `git log` mostrar uma linha reta sequencial de commits (comprovando o sucesso do `git rebase`).
 - [] O Pull Request constar na página do repositório principal do instrutor aguardando aprovação.
-